Saiyed Shizain | CU24250233 | Roll: 48
BTech CSE (A) | 3rd Yr - 5th Sem | 2024-2028

Full-Stack Web Development — BTCS303P

LAB SHEET — 3: React Components & State (Todo App)

Task 3.1 — Project setup with Vite

Aim. Scaffold a React app with Vite, install dependencies, and run the dev server.

Theory. Vite serves native ES modules in dev and bundles with Rollup for production; npm tracks deps in package.json.

```
$ npm create vite@latest react-todo -- --template react
$ cd react-todo && npm install && npm run dev
```

```
> react-todo@1.0.0 build
> vite build
vite v5.4.21 building for production...
✓ 29 modules transformed.
✓ built in 584ms
dist/index.html  0.42 kB
dist/assets/index-VlMJ6s5a.js  143.41 kB
```

Conclusion. Production bundle built successfully. — Saiyed Shizain

---

Task 3.2 — Component tree and state ownership

Aim. Split the UI into App → AddTaskForm + TaskList → TaskItem with state owned by the parent.

Theory. Lifting state up keeps one source of truth; children are controlled via props callbacks; tasks persist in localStorage.

```
react-todo/
├── src/
│   ├── components/AddTaskForm.jsx, TaskList.jsx, TaskItem.jsx
│   ├── App.jsx, App.css, main.jsx
├── index.html
└── package.json

App
├── AddTaskForm (input + Add button)
└── TaskList → TaskItem × n
```

Source — src/App.jsx

Aim. Parent component: useState tasks, localStorage persistence, add/toggle/delete handlers.

```
import { useEffect, useState } from "react";
import AddTaskForm from "./components/AddTaskForm";
import TaskList from "./components/TaskList";
import "./App.css";

// Lab Sheet 3 — Saiyed Shizain: parent component owning task state,
// persisted to localStorage so tasks survive a page reload.
function App() {
  const [tasks, setTasks] = useState(() => {
    const saved = localStorage.getItem("react-tasks");
    return saved ? JSON.parse(saved) : [];
  });

  useEffect(() => {
    localStorage.setItem("react-tasks", JSON.stringify(tasks));
  }, [tasks]);
```

```
  const addTask = (text) => {
    const newTask = { id: Date.now(), text, completed: false };
    setTasks((prev) => [...prev, newTask]);
  };

  const toggleTask = (id) => {
    setTasks((prev) =>
      prev.map((task) =>
        task.id === id ? { ...task, completed: !task.completed } : task
      )
    );
  };

  const deleteTask = (id) => {
    setTasks((prev) => prev.filter((task) => task.id !== id));
  };

  return (
    <div className="app">
      <div className="todo-card">
        <h1>React Todo App</h1>
        <p className="subtitle">Component-based task management — Saiyed Shizain</p>
        <AddTaskForm onAddTask={addTask} />
        <TaskList tasks={tasks} onToggle={toggleTask} onDelete={deleteTask} />
      </div>
    </div>
  );
}

export default App;
```

Source — src/components/AddTaskForm.jsx

Aim. Controlled form child with empty-input validation error.

```
import { useState } from "react";

// Controlled input child: validates locally, reports text upward via onAddTask.
function AddTaskForm({ onAddTask }) {
  const [text, setText] = useState("");
  const [error, setError] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();
    const value = text.trim();
    if (!value) {
      setError("Please enter a task.");
      return;
    }
    onAddTask(value);
    setText("");
    setError("");
  };

  return (
    <form onSubmit={handleSubmit} className="add-form">
      <div className="input-row">
        <input
          value={text}
          onChange={(e) => setText(e.target.value)}
          placeholder="Enter a task"
        />
        <button type="submit">Add</button>
      </div>
      {error && <p className="error">{error}</p>}
```

```
    </form>
  );
}

export default AddTaskForm;
```

Source — src/components/TaskList.jsx

Aim. List renderer with empty-state fallback message.

```
import TaskItem from "./TaskItem";

// Renders the list (or the empty-state message) from parent-owned task data.
function TaskList({ tasks, onToggle, onDelete }) {
  if (tasks.length === 0) {
    return <p className="empty">No tasks yet. Add your first task!</p>;
  }
  return (
    <div className="task-list">
      {tasks.map((task) => (
        <TaskItem
          key={task.id}
          task={task}
          onToggle={onToggle}
          onDelete={onDelete}
        />
      ))}
    </div>
  );
}

export default TaskList;
```

Source — src/components/TaskItem.jsx

Aim. Row with completion checkbox and delete button.

```
// Single row: checkbox toggles completion, button deletes the task.
function TaskItem({ task, onToggle, onDelete }) {
  return (
    <div className="task-item">
      <label>
        <input
          type="checkbox"
          checked={task.completed}
          onChange={() => onToggle(task.id)}
        />
        <span className={task.completed ? "completed" : ""}>
          {task.text}
        </span>
      </label>
      <button onClick={() => onDelete(task.id)}>Delete</button>
    </div>
  );
}

export default TaskItem;
```

Source — src/main.jsx

Aim. StrictMode root mounting the App inside #root.

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

Source — src/App.css

Aim. Card layout, form row, task rows, completed strike-through, mobile breakpoint.

```
* { box-sizing: border-box; }
body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: #eef2f7;
}
.app {
  min-height: 100vh;
  display: grid;
  place-items: center;
  padding: 30px;
}
.todo-card {
  width: min(650px, 100%);
  background: white;
  padding: 30px;
  border-radius: 18px;
  box-shadow: 0 12px 35px rgba(0, 0, 0, .10);
}
h1 { margin-bottom: 5px; }
.subtitle, .empty { color: #6b7280; }
.input-row { display: flex; gap: 10px; margin: 25px 0 5px; }
.input-row input {
  flex: 1;
  padding: 12px;
  border: 1px solid #d1d5db;
  border-radius: 8px;
}
button {
  border: 0;
  border-radius: 8px;
  padding: 10px 16px;
  background: #4f46e5;
  color: white;
  cursor: pointer;
}
.error { color: #dc2626; }
.task-item {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 13px;
  margin-top: 10px;
  background: #f5f7fb;
  border-radius: 9px;
}
.completed { text-decoration: line-through; color: #9ca3af; }
```

```
.task-item button { background: #dc2626; }
@media (max-width: 600px) {
  .input-row { flex-direction: column; }
}
```

Validation — served production bundle

Aim. Prove the built app serves correctly.

```
$ python -m http.server 8813 (in dist/)
$ curl -s -o /dev/null -w '%{http_code}' http://127.0.0.1:8813/
200
<title>React Todo App — Saiyed Shizain</title>  (personalized title served)
```

Conclusion. Install, build and serve all verified with genuine command output. — Saiyed Shizain